

AI SYSTEMS PROJECT · FULL STACK + LLM + MCP

Explain Like I'm Me

ELIM

Complete Build Roadmap & Agent Instruction Guide

Adaptive AI · Prompt Engineering · User Modeling · RAG · MCP Integration · Feedback
Loops · Multi-Modal



Table of Contents

- 01 Project Vision & Core Concept
- 02 Why This Project Is Impressive
- 03 Complete Feature List (All 8 Phases)
- 04 Tech Stack — Every Tool Chosen & Why
- 05 Full System Architecture
- 06 Database Schema — Complete Design
- 07 Backend API — Every Endpoint
- 08 AI Concepts — Deep Implementation Details
- 09 MCP Integration — Full Plan
- 10 Unique Feature Ideas
- 11 Phase-by-Phase Build Roadmap

-
- | | |
|----|--------------------------|
| 12 | UI Pages & Component Map |
|----|--------------------------|
-
- | | |
|----|--|
| 13 | Agent Instruction Prompts — Copy-Paste Ready |
|----|--|
-
- | | |
|----|------------------|
| 14 | Testing Strategy |
|----|------------------|
-
- | | |
|----|---|
| 15 | Deployment Plan & Environment Variables |
|----|---|
-
- | | |
|----|------------------------------|
| 16 | Resume & Viva Talking Points |
|----|------------------------------|
-

01 — Project Vision & Core Concept

ELIM (Explain Like I'm Me) is an adaptive AI learning system that generates personalised explanations based on each user's unique learning style, knowledge level, and history — then evolves over time based on their feedback. Unlike generic AI tools that give every user the same answer, ELIM builds a model of the learner and gets smarter about how to explain things to that specific person.

The Core Problem

Most AI explanation tools wrap ChatGPT in a UI and call it done. ELIM's differentiator is the adaptation loop — the system literally learns how each person learns.

- Every user gets the same generic explanation from standard AI tools
- No memory between sessions — feedback is lost, preferences reset
- No awareness of what the user already knows or how they learn best
- Passive reading of AI output causes shallow retention vs active personalised learning

Three Core Layers — Never Skip L3

L1 — User Modeling	Who is the learner? Captures preferred explanation style (analogy / step-by-step / code-first), difficulty level 1–5, topic history, and style_weights — a live map that evolves with every session.
L2 — Explanation Generator	How to explain? Dynamically constructs the LLM prompt at runtime by injecting user profile data, MCP-retrieved document chunks (RAG), style constraints, and prior topic context. This is the core intelligence layer.
L3 — Feedback & Adaptation Loop	Did it work? After each explanation the user rates it. The rating updates style_weights for that user. Over sessions the system auto-selects the best style. THIS is the differentiator — without L3 it's just a wrapper.

02 — Why This Project Is Impressive

This section explains exactly what makes ELIM stand out in interviews, vivas, and on a resume. Know these points cold.

Not a wrapper	Most student AI projects call an LLM with a fixed prompt. ELIM has a dynamic prompt pipeline, a user model that updates from feedback, and a RAG system pulling from live MCP sources. That is system design.
Measurable adaptation	The <code>style_weights</code> map is concrete and demonstrable. You can show before/after weights, graph how they changed, and explain exactly why. Examiners love this.
MCP integration	Connecting live external data sources (Google Drive, Notion, GitHub) via Model Context Protocol is a 2025-era technique that most developers haven't implemented. It makes RAG grounded in user-owned data.
Multi-modal output	Analogy explanations + step-by-step breakdowns + code examples + Mermaid diagrams + TTS audio. Demonstrates understanding of different learning modalities.
Production architecture	Docker, JWT auth, async FastAPI, Celery workers, Redis caching, vector DB, S3 storage — this is a real system, not a prototype.

03 — Complete Feature List (All 8 Phases)

Phase 1 — MVP

- Topic input field (free text, any subject)
- Manual learning style selector: Analogy / Step-by-Step / Code-First
- Manual difficulty selector: Beginner / Intermediate / Advanced / Expert (1–5)
- Single explanation generated via LLM API (Claude sonnet)
- Dynamic prompt template: injects topic + style + difficulty at runtime
- Explanation displayed with copy-to-clipboard button
- Follow-up question shown below explanation to prompt active engagement
- Basic FastAPI REST backend with POST /explain/generate endpoint
- React 18 + Vite + Tailwind CSS frontend
- Docker Compose local dev setup (FastAPI + MongoDB containers)
- GET /health endpoint: verifies DB and API connectivity

Phase 2 — User System

- User registration: email + password, bcrypt hashed
- JWT access token (15 min) + refresh token (30 days)
- POST /auth/register, /auth/login, /auth/refresh, /auth/logout endpoints
- JWT middleware protecting all non-auth routes
- MongoDB users collection created on registration
- Default style_weights map: {analogy:0.33, step-by-step:0.33, code:0.34}
- User preferred_style and difficulty_level auto-injected into prompt at login
- GET /profile — returns full user document
- PUT /profile/style — manual preference override
- React auth pages: Register, Login, protected routes via React Router
- Profile page: show current preferences and style_weights as bar chart
- Zustand global state for authenticated user object

Phase 3 — Feedback Loop

- Every explanation saved to MongoDB history collection
- Fields: topic, style_used, difficulty_used, prompt_snapshot, explanation, feedback_score
- POST /feedback/rate — receives {history_id, score: -1|0|1, star_rating?: 1-5}
- style_weights update formula: $\text{new_weight} = 0.8 \times \text{old} + 0.2 \times \text{signal}$, then normalise to sum=1
- time_to_rate_sec logged: fast positive rating = strong signal
- GET /profile/history — paginated explanation history
- GET /feedback/summary — per-style win rates and 30-day trend

- React History Dashboard: table of past topics, styles, scores
- Thumbs up / thumbs down buttons on every explanation card
- Optional star rating widget (collapsible, 1–5 stars)
- Auto-select best style: `argmax(style_weights)` used as default on next login
- Profile page chart updates live after each rating

Phase 4 — Multi-Style Output

- `POST /explain/multi-style` — fires 3 parallel async LLM calls (`asyncio.gather`)
- Returns `{analogy, step_by_step, code_based}` in single response
- Tab switcher UI: 3 labelled tabs, one per style
- Per-tab thumbs up/down rating — each style rated independently
- Multi-style ratings carry 2× weight in `style_weights` update (stronger signal)
- Side-by-side comparison mode: show any two styles simultaneously
- Track which style "wins" per topic per user — logged in history
- LangSmith tracing enabled for all parallel calls

Phase 5 — MCP Knowledge Integration

- Anthropic Python SDK with MCP server support enabled
- `MCPManager` class: holds authenticated clients for all enabled MCP sources
- Google Drive MCP: OAuth2 connect, index lecture notes + PDFs + slides
- Notion MCP: OAuth2 connect, index personal notes + study databases
- GitHub MCP: PAT or OAuth, index READMEs + code files + wikis
- Slack MCP: OAuth, surface study group discussions and pinned resources
- Web Search MCP: built-in fallback (`web_search_20250305`), max 3 uses
- `asyncio.gather()` queries all enabled sources simultaneously — no serial waiting
- Deduplication by SHA-256 hash of first 200 chars of each retrieved chunk
- `POST /mcp/connect/:source` — OAuth callback, store encrypted token
- `GET /mcp/sources` — returns list with connection status per source
- `DELETE /mcp/disconnect/:source` — remove token, stop querying
- Source citations shown inline in explanation: "From your CS3450 notes:"
- React Knowledge Sources settings page: toggle each source, OAuth buttons
- Redis cache: 1-hour TTL per (`user_id`, `topic`, `source`) triple
- Fallback chain: personal docs → course content → web search

Phase 6 — Full RAG Pipeline

- ChromaDB vector store (local dev) / Pinecone (cloud prod)
- Embedding model: `text-embedding-3-small` (OpenAI, 1536 dimensions)

- Chunk size: 500 tokens with 50-token overlap to preserve context at boundaries
- Cosine similarity search: top-5 chunks, minimum score threshold 0.72
- Source trust weighting: personal docs 1.0, course content 0.85, web search 0.60
- Metadata stored per chunk: source, file name, date, section, user_id
- Celery worker: incremental indexing — checks Drive/Notion daily for new files
- POST /mcp/index — manual re-index trigger with optional force flag
- GET /mcp/index/status — returns job progress (% complete, files indexed)
- GET /mcp/indexed-docs — lists all indexed documents per source
- PromptBuilder injects top-5 chunks as {{rag_context}} block
- LLM instructed: "Use only the following context. Cite your source inline."
- Collapsible "Sources used" section in React UI below explanation

Phase 7 — Multi-Modal & Chat

- POST /explain/audio — queues ElevenLabs/OpenAI TTS job via Celery
- Generated audio files stored in AWS S3 / Cloudflare R2
- React audio player: inline playback, speed control (0.75×–2×), download button
- POST /explain/diagram — LLM generates Mermaid.js diagram source code
- Mermaid.js renders diagram client-side in React — download as PNG
- POST /explain/followup — chat mode, preserves last 5 turns via LangChain ConversationBufferMemory
- React chat thread UI: message history, input box, "Start conversation" trigger
- Voice input: microphone button → OpenAI Whisper API transcription → topic field
- DALL-E 3 visual metaphors: abstract concept → AI-generated illustrative image
- Export explanation: download as PDF or Markdown file

Phase 8 — Unique Advanced Features

- Learning Pace Detector: time_to_rate_sec rolling average auto-adjusts difficulty_level
- Concept Dependency Graph: embedding similarity reveals prerequisite topics to learn first
- D3.js / react-flow interactive dependency graph rendered in UI
- Socratic Mode: LLM asks guided questions instead of explaining — toggle in UI
- Confusion Detector: NLP + LLM classifier on followup text, auto-regenerates if confused
- Spaced Repetition: SuperMemo SM-2 algorithm, Celery beat scheduler, in-app reminders
- Peer Style Matching: anonymous cluster of users with similar style_weights — borrow winning explanations
- Explanation Diff View: side-by-side comparison of same topic across different sessions
- Topic Recommendation Engine: cluster topic_history embeddings, suggest 3 "what to learn next"
- Sentry error tracking (frontend + backend)
- LangSmith prompt tracing and quality evaluation dashboard
- Full README.md with setup guide, architecture diagram, demo video link

04 — Tech Stack — Every Tool Chosen & Why

Frontend

React 18 (Vite)	Core UI framework. Vite gives instant HMR and fast builds.
Tailwind CSS v3	Utility-first styling — build beautiful UI without custom CSS files.
React Router v6	Client-side routing for all pages with protected route guards.
Framer Motion	Page transitions, loading skeletons, explanation fade-in animations.
React Query (TanStack)	Server state management, intelligent caching, background refetching.
Zustand	Client state: authenticated user, current explanation, preferences.
Mermaid.js	Client-side rendering of LLM-generated diagram source code.
Axios	HTTP client with request/response interceptors for JWT auto-refresh.
React Hot Toast	Non-blocking feedback notifications (rating saved, source connected).
D3.js / react-flow	Concept dependency graph visualisation in Phase 8.

Backend

Python 3.11+	Primary backend language. Required for async features and type hints.
FastAPI	Async REST framework. Auto-generates /docs Swagger UI. Fast.
Pydantic v2	Request/response validation and serialisation. Type-safe throughout.
Motor (async pymongo)	Non-blocking async MongoDB driver — works natively with FastAPI.
python-jose	JWT creation, signing, and verification.
Passlib + bcrypt	Industry-standard password hashing. Never store plaintext passwords.
asyncio + httpx	Parallel async HTTP calls for MCP queries and LLM fallbacks.
Celery + Redis	Background task queue: indexing, TTS generation, spaced repetition.
python-dotenv	Environment variable management from .env files.
cryptography (Fernet)	AES-256 symmetric encryption for MCP OAuth token storage.

AI / LLM / MCP

Anthropic Python SDK	Primary LLM: claude-sonnet-4-20250514. Also handles MCP server calls.
OpenAI SDK	text-embedding-3-small (embeddings), TTS, Whisper (voice), DALL-E 3.

LangChain	PromptTemplate, LLMChain, ConversationBufferMemory, OutputParser.
LangSmith	Prompt tracing, evaluation, debugging dashboard for all LLM calls.
ElevenLabs API	High-quality natural TTS voice for audio explanations (Phase 7).
Anthropic MCP SDK	MCP server connections to Google Drive, Notion, GitHub, Slack.

Database & Storage

MongoDB Atlas	Primary DB: users, history, sessions, MCP index metadata.
ChromaDB (local)	Vector store for embeddings in development environment.
Pinecone (cloud)	Production vector store — managed, scalable, per-user namespacing.
Redis (Upstash)	Celery broker, MCP response caching (1hr TTL), rate limiting.
AWS S3 / Cloudflare R2	Audio file storage for TTS outputs. R2 = no egress fees.

DevOps & Tooling

Docker + docker-compose	Local dev: all services containerised. One command to start everything.
GitHub Actions	CI pipeline: lint (ruff, eslint), test (pytest, vitest), build on every push.
Railway.app	Backend deployment: FastAPI + Celery workers, auto-deploy from GitHub.
Vercel	Frontend deployment: React app, automatic preview URLs per PR.
Sentry	Error tracking, performance monitoring, alert on production errors.
Bruno / Postman	API testing and endpoint documentation during development.

05 — Full System Architecture

ELIM has 9 interconnected components. Every explanation request flows through at least 5 of them. Understand this map before writing any code.

Component Map

React Frontend	All UI: topic input, style selector, tabs, audio player, diagram renderer, profile, history dashboard, MCP settings. Communicates with backend via Axios + React Query.
FastAPI Backend	Core API. Validates requests, orchestrates MCPManager + RAGPipeline + PromptBuilder + LLM call. All routes async. JWT middleware. Returns explanation JSON.
MCPManager	Holds authenticated Anthropic SDK clients per MCP source. <code>asyncio.gather()</code> fires all enabled sources in parallel. Deduplicates results. Returns <code>[[text, source, url]]</code> .
RAGPipeline	Receives MCP docs. Chunks (500t/50t overlap). Embeds (text-embedding-3-small). Stores in ChromaDB. Cosine similarity search returns top-5 chunks with trust-weighted ranking.
PromptBuilder	LangChain PromptTemplate. Injects: topic, style, difficulty, prior_topics, rag_context, max_words. Produces final LLM input. Different templates for each mode.
LLM Service	Claude (primary) / GPT-4o (fallback). Returns structured JSON: <code>{explanation, diagram_code, followup_question, sources_cited}</code> . Traced in LangSmith.
User Profile Engine	Reads/writes MongoDB user documents. Computes style_weights from feedback. Provides profile context. Triggers difficulty auto-adjustment via Pace Detector.
Feedback Engine	Receives rating events. Applies weighted update to style_weights. Normalises to sum=1. Logs to history. Triggers next-session style auto-selection.
Celery Workers	Background tasks: daily MCP re-indexing, TTS generation queue, spaced repetition scheduler, batch embedding jobs. Redis as broker.

Data flow for every explanation: React → POST /explain/generate → MCPManager (parallel) → RAGPipeline → PromptBuilder → LLM → FastAPI → React display. On rating: React → POST /feedback/rate → Feedback Engine → MongoDB update.

File & Folder Structure

```
elim/ ├── frontend/ (React + Vite) |   ├── src/ |   |   ├── pages/ (Home, Profile, History, Settings, Auth) |   |   ├── components/ (ExplainCard, StyleTabs, AudioPlayer, DiagramView, ChatThread) |   |   ├── hooks/ (useExplain, useFeedback, useMCP) |   |   └── store/ (useAuthStore, useExplainStore) |   └── backend/ (FastAPI) |   |   ├── routers/ (auth, explain, feedback, profile, mcp) |   |   ├── services/ (mcp_manager, rag_pipeline, prompt_builder, llm_service, feedback_engine) |   |   ├── models/ (user, history, embedding, session) |   |   └── workers/ (indexer, tts_worker, spaced_rep) |   └── docker-compose.yml |   └── .env.example
```

06 — Database Schema — Complete Design

Collection: users

Field	Type	Description
<code>_id</code>	ObjectId	MongoDB auto-generated primary key
<code>email</code>	String (unique, indexed)	Used for login. Must be unique.
<code>password_hash</code>	String	bcrypt hash. NEVER store plaintext.
<code>name</code>	String	Display name shown in UI
<code>preferred_style</code>	String	analogy step-by-step code-based auto
<code>difficulty_level</code>	Number 1–5	1=beginner, 5=expert. Auto-adjusted by Pace Detector.
<code>style_weights</code>	Map<String,Float>	{"analogy":0.33,"step-by-step":0.33,"code":0.34} — live, normalised to 1.0
<code>enabled_mcp_sources</code>	Array<String>	["gdrive","notion","github","slack","web"]
<code>mcp_tokens</code>	Map<String,String>	Fernet-encrypted OAuth tokens per source
<code>topic_history</code>	Array<String>	Last 50 topics — seeds RAG context queries
<code>spaced_rep_queue</code>	Array<Object>	[{topic, next_review_at, interval_days, ease_factor}]
<code>created_at</code>	Date	Account creation timestamp
<code>last_active</code>	Date	Updated on every explanation request

Collection: history

Field	Type	Description
<code>_id</code>	ObjectId	Auto-generated
<code>user_id</code>	ObjectId (indexed)	Reference to users._id
<code>topic</code>	String	Topic the user requested an explanation for
<code>style_used</code>	String	Which style was generated (analogy/step-by-step/code)
<code>difficulty_used</code>	Number 1–5	Difficulty level at time of generation
<code>prompt_snapshot</code>	String	Full prompt sent to LLM — for debugging and Diff View
<code>explanation</code>	String	Full LLM explanation output text
<code>diagram_code</code>	String	Mermaid.js diagram source code (if generated)

audio_url	String	S3/R2 URL of TTS audio file (if generated)
mcp_sources_used	Array<String>	Which MCP sources contributed context
rag_chunks	Array<Object>	Top-5 chunks used: [{text, source, score, file}]
feedback_score	Number	-1 (thumbs down), 0 (no rating), +1 (thumbs up)
star_rating	Number null	1–5 optional granular rating
time_to_rate_sec	Number	Seconds between display and rating — pace signal
created_at	Date	Timestamp of explanation generation

Collection: embeddings

Field	Type	Description
_id	ObjectId	Auto-generated
user_id	ObjectId (indexed)	Owner — embeddings are NEVER shared between users
source	String	gdrive notion github slack web
source_url	String	Original document URL or Drive file ID
chunk_text	String	Raw text of this 500-token chunk
embedding	Array<Float>	1536-dimensional vector from text-embedding-3-small
metadata	Object	{filename, page, section, date_indexed, word_count}
chroma_id	String	ID in ChromaDB/Pinecone for fast retrieval lookup
indexed_at	Date	Timestamp when this chunk was embedded and stored

Collection: sessions

Field	Type	Description
_id	ObjectId	Auto-generated
user_id	ObjectId	Reference to users._id
refresh_token	String (hashed)	SHA-256 hash of the refresh token issued
device_info	String	User-agent fingerprint for security auditing
expires_at	Date	30-day expiry — delete expired sessions via TTL index

07 — Backend API — Every Endpoint

All endpoints require Bearer JWT in Authorization header unless marked as Public. FastAPI auto-generates Swagger docs at /docs.

Auth (Public) Endpoints

Method	Route	Request Body / Query	Response / Action
POST	/auth/register	{email,password,name}	Creates user with default weights. Returns {access_token,refresh_token}.
POST	/auth/login	{email,password}	Verifies password. Returns {access_token,refresh_token,user}.
POST	/auth/refresh	{refresh_token}	Issues new access_token without re-login.
POST	/auth/logout	{refresh_token}	Invalidates refresh token in sessions collection.

Explanation Endpoints

Method	Route	Request Body / Query	Response / Action
POST	/explain/generate	{topic, style?, difficulty?}	Main endpoint. MCP → RAG → Prompt → LLM. Returns {explanation, followup, sources, history_id}.
POST	/explain/multi-style	{topic, difficulty?}	3 parallel LLM calls. Returns {analogy, step_by_step, code_based, history_ids}.
POST	/explain/followup	{history_id, question}	Chat mode. Maintains last 5 turns via ConversationBufferMemory.
POST	/explain/audio	{history_id}	Queues TTS Celery job. Returns {job_id}. Poll /explain/audio/{job_id} for {audio_url}.
POST	/explain/diagram	{history_id}	Re-prompts LLM for Mermaid.js diagram code. Returns {diagram_code}.
POST	/explain/socratic	{topic, difficulty?}	Socratic mode: LLM asks guided questions instead of explaining.

Feedback Endpoints

Method	Route	Request Body / Query	Response / Action
POST	/feedback/rate	{history_id, score, star_rating?}	Saves rating. Triggers style_weights update. Returns {updated_weights}.

GET	/feedback/summary	?days=30	Per-style win rates and trend chart data for the authenticated user.
-----	-------------------	----------	--

User Profile Endpoints

Method	Route	Request Body / Query	Response / Action
GET	/profile	—	Full user document including style_weights, difficulty, topic_history.
PUT	/profile/style	{preferred_style, difficulty_level}	Manual override of user preferences.
GET	/profile/history	?page=1&limit=20&topic=	Paginated explanation history with filtering.
DELETE	/profile/history/:id	—	Delete a single history entry and its embedding if indexed.
GET	/profile/spaced-rep	—	Returns topics due for spaced repetition review today.

MCP Sources Endpoints

Method	Route	Request Body / Query	Response / Action
GET	/mcp/sources	—	All MCP sources with {name, connected, last_indexed, doc_count}.
POST	/mcp/connect/:source	{oauth_code, redirect_uri}	Complete OAuth flow. Encrypt and store token. Returns {connected:true}.
DELETE	/mcp/disconnect/:source	—	Remove token from DB. Stop querying this source.
POST	/mcp/index	{source, force?}	Trigger manual Celery indexing job. Returns {job_id}.
GET	/mcp/index/status	?job_id=	Returns {progress_pct, files_indexed, status}.
GET	/mcp/indexed-docs	?source=&page=1	Paginated list of indexed documents for a source.

Admin / Debug Endpoints

Method	Route	Request Body / Query	Response / Action
GET	/health	—	Public. Returns {status, db, redis, mcp_reachable}.
GET	/debug/prompt/:history_id	Admin JWT required	Returns prompt_snapshot for debugging an explanation.

08 — AI Concepts — Deep Implementation Details

Dynamic Prompt Engineering

- LangChain PromptTemplate with runtime variables: {topic}, {style}, {difficulty}, {prior_knowledge}, {rag_context}, {max_words}
- Positive constraints: "Use analogies from everyday life" / "Walk through step by step" / "Show working code in Python"
- Negative constraints: "Avoid technical jargon", "Do not assume maths knowledge", "No formulas"
- RAG block: "Use ONLY the following material from the user's personal sources. Cite each source inline: {rag_context}"
- Closing instruction: "End with one follow-up question to check understanding"
- Separate prompt templates for: single-style, multi-style, followup/chat, diagram, socratic mode
- All prompts traced in LangSmith — compare quality across versions

User Modeling & Personalization

- style_weights Map starts equal: {analogy:0.33, step-by-step:0.33, code:0.34}
- Update formula per rating: $\text{new_weight} = (0.8 \times \text{old_weight}) + (0.2 \times \text{feedback_signal})$
- feedback_signal: +1.0 (thumbs up), 0.0 (no rating), -1.0 (thumbs down)
- Multi-style ratings carry 2× signal: user rated after seeing all 3 styles — higher confidence
- After update: normalise all weights so they sum to exactly 1.0
- Auto-select style: $\text{argmax}(\text{style_weights})$ chosen as default on next session start
- difficulty_level auto-adjusted by Pace Detector: fast+positive → increment, slow+negative → decrement
- topic_history (last 50) included in RAG query to surface related prior explanations as context

Retrieval-Augmented Generation (RAG)

- Embedding model: text-embedding-3-small — 1536 dimensions, cost-efficient
- Chunk size: 500 tokens with 50-token overlap. Overlap preserves context at chunk boundaries.
- Vector store: ChromaDB (local dev, zero config) / Pinecone (prod, managed, per-user namespace)
- Retrieval: cosine similarity, top-5 chunks, score threshold ≥ 0.72 (discard low-relevance)
- Source trust weighting: personal docs $\times 1.0$, course material $\times 0.85$, web $\times 0.60$
- Final ranking = cosine_score $\times$ trust_weight — personal notes beat generic web results
- Chunk metadata stored: source, filename, section, date_indexed — displayed as citation in UI

- Incremental indexing via Celery beat: checks Drive/Notion daily for new/modified files

Adaptive Heuristics — Feedback Loop

- NOT full Reinforcement Learning — weighted moving average heuristic (simpler, demonstrable, impressive)
- Rolling window: last 5 sessions used for pace detection, last 20 for weight trend chart
- time_to_rate_sec: < 10s positive = very strong signal. > 60s = uncertain, weight at 0.5×
- Confusion Detector: if followup question contains confusion keywords → re-generate with different style + reduce weight of style used
- Multi-style win tracking: the style the user rated highest across comparison mode = winning style, extra weight update
- Trend analytics endpoint: graph weight evolution over 30 days — show user how system learned about them

LangChain Orchestrati on

- PromptTemplate: define template string + input_variables list per mode
- LLMChain: template → Claude API call → OutputParser (extract JSON fields)
- ConversationBufferMemory: stores last 5 turns for chat/followup mode — included in next prompt
- Multi-style: three independent LLMChains run via asyncio.gather — parallel, not sequential
- OutputParser: expects LLM to return JSON: {explanation, diagram_code, followup, citations}
- Error handling: if Claude fails → retry once → fallback to GPT-4o → return error if both fail
- LangSmith: log every chain run (inputs, outputs, latency, token count) for quality monitoring

Semantic Memory — Embedding s

- All MCP-retrieved docs embedded on first query and cached — no re-embedding on subsequent requests
- topic_history embeddings computed and stored — enables concept dependency detection
- Concept Dependency Graph: new topic embedding compared to all history embeddings. Topics with high cosine similarity are likely prerequisites.
- Peer Matching: k-means cluster on style_weights vectors. Users in same cluster share winning explanation styles (text only, never PII).
- Spaced Repetition uses explanation embedding to generate a re-explanation from a different angle on review day

09 — MCP Integration — Full Plan

MCP (Model Context Protocol) is an open standard by Anthropic that lets ELIM connect to external data sources through a standardised interface. Instead of static pre-indexed content, the RAG pipeline gets live, user-owned material every time.

What MCP Adds to ELIM

Without MCP	Generic web content indexed once. Stale. Same for every user. Low relevance.
With MCP	Live retrieval from user's own Drive, Notion, GitHub. Fresh, relevant, personal. Explanations cite "your CS3450 lecture PDF".

MCP Sources — Priority & Details

Priority	Source	MCP URL	Description & Trust Weight
1st	Google Drive	https://drivemcp.googleapis.com/mcp/v1	Lecture PDFs, slides, textbook chapters. Trust: 1.0 (user's own material = highest weight in RAG ranking).
2nd	Notion	https://mcp.notion.com/mcp	Personal study notes, knowledge bases, flashcard DBs. Trust: 1.0
3rd	GitHub	https://api.githubcopilot.com/mcp/	READMEs, code files, algorithm implementations. Trust: 0.85
4th	Slack	https://mcp.slack.com/mcp	Study group discussions, Q&A threads, pinned resources. Trust: 0.85
5th (fallback)	Web Search	Built-in: <code>web_search_20250305</code>	Wikipedia, MDN, ArXiv, GeeksForGeeks. Max 3 uses. Trust: 0.60

MCPManager Class — Implementation

```
File: backend/services/mcp_manager.py class MCPManager:
    def __init__(self, enabled_sources: list[str], encrypted_tokens: dict):
        self.sources = enabled_sources
        self.tokens = {k: fernet.decrypt(v) for k,v in encrypted_tokens.items()}
    async def search(self, topic: str) -> list[dict]:
        tasks = [self._query_source(src, topic) for src in self.sources]
        results = await asyncio.gather(*tasks, return_exceptions=True)
        return self._deduplicate(flatten(results))
    async def _query_source(self, source: str, topic: str):
        if source == "web":
            return await self._web_search(topic)
        client = anthropic.Anthropic()
        resp = await client.beta.messages.create(
            model="claude-sonnet-4-20250514",
            mcp_servers=[{"type": "url", "url": MCP_URLS[source], "name": source, "authorization_token": self.tokens[source]}],
            messages=[{"role": "user", "content": f"Find all content about: {topic}"}]
        )
        return parse_mcp_response(resp)
```

MCP Data Flow — Step by Step

1. User submits topic → POST /explain/generate received by FastAPI
2. Load user profile from MongoDB: enabled_mcp_sources, mcp_tokens (decrypt with Fernet)
3. Instantiate MCPManager(enabled_sources, tokens)
4. MCPManager.search(topic) → asyncio.gather() fires all sources in parallel
5. Each source returns raw document text + metadata (filename, URL, date)
6. Deduplicate by SHA-256 of first 200 chars — remove duplicate chunks from overlapping sources
7. Pass cleaned docs to RAGPipeline.process(docs, user_id)
8. RAGPipeline chunks into 500-token segments with 50-token overlap
9. Embed each chunk via text-embedding-3-small — batch API call for efficiency
10. Upsert vectors into ChromaDB under collection namespace f"user_{user_id}"
11. Similarity search: embed topic query → cosine search → top-5 chunks (score ≥ 0.72)
12. Apply source trust weights: rank = cosine_score × trust_weight[source]
13. Format top-5 as rag_context string with source citations
14. PromptBuilder injects rag_context + user profile into LangChain template
15. Claude generates grounded personalised explanation with inline citations
16. Response returned with explanation + sources array → displayed in React UI

Security Rules — MCP Token Handling

- All OAuth tokens encrypted with Fernet (AES-128-CBC) before MongoDB storage
- Tokens never appear in logs — logging middleware has mcp_tokens in exclude list
- OAuth scopes: read-only where available (drive.readonly, GitHub repo read)
- MCP server URLs validated against hardcoded allowlist before every connection
- Per-user rate limit: max 10 MCP API calls per /explain/generate call
- Tokens auto-refreshed by Anthropic SDK when expiring — no manual refresh logic needed
- On user account deletion: all encrypted tokens purged from DB immediately

React UI — MCP Settings Page (/settings/sources)

- Source list: Google Drive, Notion, GitHub, Slack, Web Search
- Each source card shows: icon, name, connection status badge (Connected/Not connected)
- "Connect" button → OAuth redirect → callback to POST /mcp/connect/:source
- "Disconnect" button → confirmation modal → DELETE /mcp/disconnect/:source
- Indexed document count shown per connected source
- "Re-index now" button → POST /mcp/index → shows progress bar from GET /mcp/index/status
- Toggle: enable/disable each source per-query without disconnecting

10 — Unique Feature Ideas

These features go beyond the spec and are what make ELIM memorable. Implement them in Phase 8 or as stretch goals.

Learning Pace Detector

Tracks `time_to_rate_sec` for each explanation. Rolling average over last 5 sessions. If user rates quickly (+ve) → auto-increment `difficulty_level`. If slowly or negatively → decrement. No slider interaction needed — difficulty adapts invisibly. Stored per-user in MongoDB.

Implementation: `difficulty_adjuster.py` in Celery worker. Runs after every `POST /feedback/rate`.

Concept Dependenc y Graph

When a user asks about "Binary Trees", compute embedding similarity between the topic and all entries in `topic_history`. Topics with cosine similarity > 0.8 are related — ones the user hasn't studied yet are prerequisites. Show an interactive graph (react-flow) before the explanation: "You should understand these first."

Implementation: `dependency_graph.py` service. `GET /profile/dependencies?topic=` endpoint. React: `DepGraph` component using `react-flow`.

Socratic Mode

Toggle in UI. Changes prompt template entirely: "Do NOT explain {topic}. Instead, ask the user 3 progressively guiding Socratic questions that lead them to discover the concept themselves." Each question becomes a chat turn. Research shows this improves retention vs passive reading.

Implementation: `socratic_template` in `prompt_builder.py`. `POST /explain/socratic` endpoint. React: `SocraticChat` component.

Confusion Detector

Classify follow-up questions using LLM: "Does this follow-up question (text) indicate confusion or understanding?" If confusion detected: automatically re-generate the explanation using a different style, reduce the `style_weight` of the style that failed, and show a gentler version.

Implementation: `confusion_classifier.py`. Runs inside `POST /explain/followup` handler. Triggers auto-regen if `confusion_score > 0.7`.

Explanatio n Diff View

Show how the same topic's explanation changed as `style_weights` evolved. Fetch two history entries for the same topic from different dates. Highlight sentence-level differences. User sees: "This is how ELIM explained Quicksort to you 3 weeks ago vs today." Demonstrates the system learning about them.

Implementation: GET /profile/history/diff?topic=quicksort&dates=2025-01-01,2025-02-01. React: DiffView component.

Spaced Repetition Reminders

Uses SuperMemo SM-2 algorithm. After each positively-rated explanation, calculate next review date based on `ease_factor` and current interval. Celery beat task runs daily, finds due reviews, sends in-app notification or email. Re-explanation adapts to current `style_weights` at review time.

Implementation: spaced_rep.py Celery task. spaced_rep_queue array in user document. GET /profile/spaced_rep endpoint.

Peer Style Matching

Privacy-safe anonymous clustering. K-means on `style_weights` vectors across all users. Users in the same cluster likely learn similarly. When a new user has only 2 sessions, borrow winning explanation texts (not user data) from similar users with 50+ sessions. Cold-start problem solved.

Implementation: peer_matcher.py service. Runs as weekly Celery batch. Anonymised cluster_id stored per user. Only explanation text shared, never PII.

Voice Input + Output

Microphone button in React → MediaRecorder API → WAV blob → POST /voice/transcribe → OpenAI Whisper API → transcribed text fills topic field. TTS output: ElevenLabs or OpenAI TTS converts explanation → MP3 → S3 → streamed audio player with speed control.

Implementation: POST /voice/transcribe endpoint. POST /explain/audio endpoint with Celery TTS worker. React: VoiceInput component, AudioPlayer component.

Topic Recommendation Engine

After each session: cluster `topic_history` embeddings → find centroid → compute distance from centroid to all topics in a curated CS/science knowledge graph → surface the 3 closest unseen topics as "What to learn next". Shown on home page after each explanation.

Implementation: topic_recommender.py service. GET /profile/recommendations endpoint. Knowledge graph: JSON file of topic embeddings for 500 common concepts.

Explanation Quality Scorer

After LLM generates explanation: run a second LLM call (or rule-based check) scoring it on: clarity (1-5), accuracy (1-5), style adherence (1-5). If average < 3.0, auto-retry with adjusted temperature. Log quality scores for LangSmith analysis. Never show a low-quality explanation.

Implementation: `quality_scorer.py` called inside LLM service. Retry max 2 times. `quality_score` saved to history document.

11 — Phase-by-Phase Build Roadmap

***RULE:** Complete all tasks in a phase and pass the test checklist before starting the next phase. Each phase produces a deployable increment.*

Phase 1	MVP — Working Prototype	Week 1
	• Create monorepo: /frontend (Vite+React) and /backend (FastAPI) at project root • Install frontend deps: react, react-dom, react-router-dom, tailwindcss, axios, zustand, @tanstack/react-query • Install backend deps: fastapi, motor, anthropic, python-jose[cryptography], passlib[bcrypt], python-dotenv, uvicorn, pydantic • Create .env.example with all variables from Section 15 • Set up docker-compose.yml: fastapi service (port 8000), mongodb service (port 27017) • Create FastAPI app with CORS middleware, /health endpoint returning {status:"ok",db:"connected"} • Create MongoDB connection module using Motor async client • Implement POST /explain/generate: receive {topic,style,difficulty}, build prompt using f-string template, call Claude API, return {explanation, followup} • Prompt template (Phase 1): "Explain {topic} to a {difficulty}/5 difficulty learner. Use {style} approach. Avoid jargon. End with one check question." • Build React pages: Home.jsx (topic input + style select + difficulty slider + submit + output display) • Connect frontend to backend: Axios instance with baseURL from env variable • ☒ TEST: enter "binary search", select analogy, difficulty 2 → explanation displayed in < 10 seconds	

Phase 2	User Auth & Profiles	Week 1–2
	• Create users collection with schema from Section 06 • POST /auth/register: hash password (bcrypt rounds=12), create user doc with default style_weights, return JWT • POST /auth/login: verify password, create session doc, return access_token (15min) + refresh_token (30day) • POST /auth/refresh: verify refresh token hash, issue new access_token • POST /auth/logout: delete session doc by refresh_token hash • JWT middleware: FastAPI dependency that extracts user_id from Bearer token on every protected route • GET /profile: return user document (exclude password_hash) • PUT /profile/style: update preferred_style and difficulty_level • Update POST /explain/generate: if authenticated, read preferred_style + difficulty from DB, inject into prompt (user overrides trump request body defaults) • Build React: Register.jsx, Login.jsx, auth context with Zustand, protected route wrapper component • Build Profile.jsx: show style_weights as horizontal bar chart (CSS widths driven by weight values)	

- ☒ TEST: register → login → generate explanation → JWT sent in header → profile style auto-injected

Phase 3**Feedback Loop & History**

Week 2

- Create history collection with schema from Section 06
- On every successful /explain/generate: save full history document, return history_id in response
- POST /feedback/rate: receive {history_id, score, star_rating?}, update history.feedback_score, call update_style_weights()
- update_style_weights(): $\text{new_weight} = 0.8 \times \text{old} + 0.2 \times \text{signal}$, then divide all weights by sum to normalise
- Record time_to_rate_sec: frontend sends timestamp of display, backend computes diff from created_at
- GET /profile/history: paginated list with ?page=1&limit=20, sorted by created_at desc
- GET /feedback/summary: aggregate per-style feedback_score averages over ?days=30
- Build React History.jsx: table of past explanations (topic, style, date, score)
- Add ThumbsRating component to explanation display: thumbs up / thumbs down buttons
- Add optional StarRating component (collapsible, 1–5 stars with labels)
- Update Profile.jsx: weights chart updates live after each rating via React Query refetch
- ☒ TEST: rate 5 explanations with analogy → profile weights should show analogy > 0.5
- ☒ TEST: next generation should auto-select analogy without user choosing it

Phase 4**Multi-Style Output**

Week 2–3

- POST /explain/multi-style: fire 3 async LLM calls via asyncio.gather — one per style prompt
- Return {analogy:{explanation,history_id}, step_by_step:{...}, code_based:{...}}
- Build React StyleTabs component: tabbed UI switching between 3 explanations
- Add individual ThumbsRating to each tab — POST /feedback/rate per style
- Update style_weights update: multi-style rating uses 2× signal weight (higher confidence)
- POST /explain/multi-style should also save time_to_rate_sec when user first rates any tab
- Add comparison mode toggle: show two styles side-by-side (grid layout, user picks two tabs)
- Track winning_style per topic in history document
- ☒ TEST: generate multi-style → all 3 tabs have content → rate each tab individually
- ☒ TEST: after 5 multi-style sessions rating code highest → code weight > 0.5 in profile

Phase 5**MCP Knowledge Integration**

Week 3

- Install anthropic SDK with MCP beta support enabled
- Create backend/services/mcp_manager.py with MCPManager class (see Section 09)
- Add enabled_mcp_sources and mcp_tokens fields to users schema and default values
- Install cryptography library for Fernet token encryption
- Implement OAuth callback handlers for Google Drive (port 8000/auth/callback/gdrive)
- Implement OAuth callback handlers for Notion (/auth/callback/notion)
- Implement GitHub PAT storage via POST /mcp/connect/github with {token} body

- POST /mcp/connect/:source: exchange OAuth code for token, encrypt, store in user.mcp_tokens
- GET /mcp/sources: return [{name, connected, last_indexed, doc_count}] for all 5 sources
- DELETE /mcp/disconnect/:source: remove token from user.mcp_tokens, set source to disconnected
- Update POST /explain/generate: instantiate MCPManager from user profile → call search() → include docs in rag_context prompt variable
- Add mcp_sources_used to history document on save
- Build React Settings.jsx with SourceCard components: icon, name, status badge, Connect/Disconnect button
- Show source citations below explanation: collapsible Sources section with chip per source
- ☒ TEST: connect Google Drive → upload a PDF about quicksort → ask about quicksort → see PDF cited in explanation
- ☒ TEST: disconnect Google Drive → generate same topic → Drive PDF no longer cited

Phase 6

Full RAG Pipeline

Week 3–4

- Install chromadb, langchain, openai (embeddings), celery, redis
- Create backend/services/rag_pipeline.py with RAGPipeline class
- Implement chunk_documents(): split by 500 tokens, 50-token overlap using tiktoken
- Implement embed_chunks(): batch call to text-embedding-3-small — batch size 100 for cost efficiency
- Implement store_embeddings(): upsert into ChromaDB collection f"user_{user_id}" with metadata
- Implement search(): embed topic → cosine similarity → top-5 chunks with score ≥ 0.72
- Implement apply_trust_weights(): multiply cosine_score $\times$ trust_weight[source], re-sort
- Set up Celery with Redis broker: celery -A backend.workers.celery app
- Create IndexingWorker Celery task: check Drive/Notion for new files daily via Celery beat
- POST /mcp/index: trigger manual Celery indexing job, return {job_id}
- GET /mcp/index/status: Redis-backed job status {progress_pct, files_done, total_files}
- GET /mcp/indexed-docs: paginated list from embeddings collection filtered by source
- Update PromptBuilder: format top-5 chunks as numbered context block with source labels
- Build React: expand Sources section to show chunk previews, progress bar for indexing
- ☒ TEST: index a PDF → ask question answered in PDF → see exact sentence cited with filename
- ☒ TEST: add new file to Drive → wait for Celery job → new content appears in explanations

Phase 7

Multi-Modal & Chat Mode

Week 4

- POST /explain/audio: validate history_id, queue Celery TTS job, return {job_id}
- TTS Celery worker: fetch explanation text from DB → call ElevenLabs/OpenAI TTS → upload MP3 to S3 → update history.audio_url → set job complete in Redis
- Set up S3 bucket or Cloudflare R2 with signed URL generation (24hr expiry)
- Build React AudioPlayer component: HTML5 audio element, play/pause, speed selector (0.75 $\times$ 1 $\times$ 1.5 $\times$ 2 $\times$), download button
- POST /explain/diagram: call LLM with diagram prompt "Generate Mermaid.js diagram code for: {topic}. Return only the raw mermaid source, no markdown fences."

- Build React DiagramView component: import and render mermaid.js, button to download as PNG via html2canvas
- POST /explain/lookup: load ConversationBufferMemory for history_id, add user question, call LLM with full history context, return {reply, followup}
- ConversationBufferMemory stored in Redis with 2-hour TTL per conversation
- Build React ChatThread component: message bubbles, input field, auto-scroll, "Start conversation" trigger below explanation
- Voice input: MediaRecorder API → WAV Blob → POST /voice/transcribe → OpenAI Whisper → fill topic field
- Add microphone icon button to topic input field — only shows in supported browsers
- ☒ TEST: generate explanation → click audio → audio plays within 10 seconds
- ☒ TEST: generate → click diagram → Mermaid flowchart renders in UI
- ☒ TEST: ask 3 follow-up questions → each answer correctly references the original topic

Phase 8**Unique Features & Polish**

Week 4–5

- Learning Pace Detector: after every /feedback/rate, compute rolling avg of time_to_rate_sec, update difficulty_level (± 1 if 3 consecutive sessions trend same direction)
- Spaced Repetition: implement SM-2 algorithm in spaced_rep.py, Celery beat daily task finds due items, update spaced_rep_queue in user doc
- Build React SpacedRep notification badge in header: count of topics due for review today
- Concept Dependency Graph: GET /profile/dependencies?topic= endpoint, compute prerequisite topics via embedding similarity, render with react-flow
- Socratic Mode: POST /explain/socratic endpoint with Socratic prompt template, build SocraticChat UI component
- Confusion Detector: inside POST /explain/followup, call confusion_classifier LLM call, if confused → auto-regen with different style + log
- Explanation Diff View: GET /profile/history/diff?topic=&dates= — fetch two history entries, compute sentence diff, build DiffView React component
- Topic Recommendation Engine: GET /profile/recommendations — cluster topic_history embeddings, return 3 closest unseen topics with "Why?" explanations
- Explanation Quality Scorer: inside LLM service, second call scores clarity+accuracy, retry if avg < 3.0
- Set up Sentry: dsn in .env, Sentry.init() in both React main.jsx and FastAPI main.py
- Set up LangSmith: LANGCHAIN_TRACING_V2=true in .env, view dashboard for all LLM call traces
- Write README.md: prerequisites, docker-compose up, environment variables, feature overview, architecture diagram
- Final QA: test all 8 phases end-to-end on a fresh database with a new user
- ☒ DEPLOY: railway up (backend), vercel --prod (frontend)

12 — UI Pages & Component Map

Route	Page Name	Key Components
/	Home / Explain	Topic input field, style selector (tabs), difficulty slider, Submit button, ExplainCard (output + ratings + audio + diagram + sources), SocraticChat toggle
/auth/login	Login	Email + password form, link to register, JWT stored in Zustand on success
/auth/register	Register	Name + email + password, submit → auto-login
/profile	My Profile	style_weights bar chart, difficulty level badge, manual style override controls, last active date
/history	History	Paginated table: topic style date rating star click to expand full explanation
/history/diff	Explanation Diff	Select two sessions for same topic → sentence-level diff view (Phase 8)
/settings/sources	Knowledge Sources	SourceCard ×5: connect/disconnect OAuth, toggle, indexing progress bar, doc count
/learn	What to Learn Next	TopicRecommendation cards (Phase 8), SpacedRep due today badge, dependency graph viewer
/compare	Style Comparison	Two-panel side-by-side explanation view, per-panel rating

13 — Agent Instruction Prompts — Copy-Paste Ready

Use these prompts verbatim with your AI coding agent. Each is self-contained with all context the agent needs. Start with Phase 1 and work forward.

Phase 1 — MVP — Agent Prompt

You are building ELIM (Explain Like I'm Me), an adaptive AI learning system. Build Phase 1: MVP prototype.

PROJECT STRUCTURE:

- /frontend — React 18 + Vite + Tailwind CSS
- /backend — Python FastAPI
- /docker-compose.yml

STEP 1 — Backend setup:

Create /backend/main.py with FastAPI app. Install: fastapi uvicorn motor anthropic python-dotenv pydantic

- Add CORS middleware allowing all origins (dev only)
- GET /health: connect to MongoDB via Motor, return {"status":"ok","db":"connected"}
- POST /explain/generate: Body schema {topic: str, style: str = "analogy", difficulty: int = 2}

Build prompt: "You are an expert teacher. Explain {topic} to a {difficulty}/5 difficulty learner (1=beginner, 5=expert). Use a {style} approach (analogy=real-world comparisons, step-by-step=numbered walkthrough, code-based=working code examples). Avoid jargon unless necessary. End with exactly one follow-up question to check understanding."

Call Anthropic claude-sonnet-4-20250514 with this prompt. Parse response.

Return: {"explanation": str, "followup": str, "style": str, "topic": str}

- Load ANTHROPIC_API_KEY from .env file using python-dotenv

STEP 2 — Frontend setup:

Create /frontend with: npm create vite@latest frontend -- --template react

Install: axios tailwindcss @tailwindcss/vite zustand @tanstack/react-query

Build src/pages/Home.jsx:

- Text input for topic (placeholder: "Enter any topic e.g. binary search")
- Button group for style selection: Analogy | Step-by-Step | Code-First (highlight active)
- Slider for difficulty 1-5 with label (1=Beginner, 5=Expert)
- Submit button that calls POST http://localhost:8000/explain/generate
- Loading spinner while fetching
- ExplainCard component: shows explanation text in a card, shows followup question in a callout box
- Copy to clipboard button on explanation card

STEP 3 — Docker:

Create docker-compose.yml with services: fastapi (Dockerfile, port 8000) and mongodb (image: mongo:7, port 27017)

Create /backend/Dockerfile: FROM python:3.11-slim, COPY requirements.txt, RUN pip install, COPY . ., CMD uvicorn main:app --host 0.0.0.0 --port 8000

DO NOT add auth yet. DO NOT add database writes yet. Focus only on working end-to-end explain flow.

Test manually: curl -X POST http://localhost:8000/explain/generate -H "Content-Type: application/json" -d '{"topic":"binary search","style":"analogy","difficulty":2}'

Phase 3 — Feedback Loop — Agent Prompt

ELIM Phase 3: Feedback Loop. Existing: Phase 1+2 working (auth, JWT, user profiles, explanation generation saving user prefs to prompt).

ADD these features:

BACKEND:

1. Ensure every POST /explain/generate saves to MongoDB history collection with fields: user_id (from JWT), topic, style_used, difficulty_used, prompt_snapshot (full prompt string), explanation (full text), feedback_score=0, star_rating=null, time_to_rate_sec=null, created_at=datetime.utcnow()
Return history_id in response alongside explanation.
2. POST /feedback/rate
Body: {history_id: str, score: int (-1|0|1), star_rating: int|None, display_time_utc: str (ISO)}
 - Fetch history document, verify it belongs to authenticated user
 - Compute time_to_rate_sec = (datetime.utcnow() - display_time_utc).seconds
 - Update history: feedback_score=score, star_rating=star_rating, time_to_rate_sec=computed
 - Call update_style_weights(user_id, style_used, score, time_to_rate_sec):
 - * signal = score (-1, 0, or +1)
 - * if time_to_rate_sec < 10 and score > 0: signal_strength = 1.2 (fast positive = confident)
 - * new_weight[style_used] = (0.8 × old_weight[style_used]) + (0.2 × signal × signal_strength)
 - * normalise: divide all weights by sum so they total 1.0
 - * update user document in MongoDB
 - Return {updated_weights: dict}
3. GET /profile/history?page=1&limit=20
Paginated, sorted by created_at desc. Return array of {_id, topic, style_used, feedback_score, star_rating, created_at}
4. GET /feedback/summary?days=30
Aggregate history for authenticated user in last N days.
Return {per_style: {analogy:{count,avg_score}, step-by-step:{...}, code-based:{...}}, best_style: str}

FRONTEND:

1. ExplainCard: add ThumbsRating component (thumbs up / thumbs down buttons)
On mount: record display_timestamp = new Date().toISOString()
On thumb click: POST /feedback/rate with {history_id, score:1|-1, display_time_utc}
Show visual confirmation after rating (toast notification)
2. Add optional star rating (1-5) below thumbs – collapsible, shows after thumb given
3. History.jsx page at /history: table with columns Topic | Style | Date | Score (star or thumbs icon)
Click row to expand full explanation text
4. Profile.jsx: style_weights shown as horizontal bars where width% = weight × 100
Bars animate when weights change (React Query refetch after rating)
5. Update /explain/generate call: after response received, set preferred_style in next call to the argmax of updated weights (read from profile GET)

Phase 5 — MCP Integration — Agent Prompt

ELIM Phase 5: MCP Knowledge Integration. Existing: auth, profiles, feedback loop, multi-style output all working.

INSTALL: `pip install anthropic[mcp] cryptography`

BACKEND:

1. Create `backend/services/mcp_manager.py`:

```
import asyncio, hashlib, anthropic
from cryptography.fernet import Fernet

MCP_URLS = {
    "gdrive": "https://drivemcp.googleapis.com/mcp/v1",
    "notion": "https://mcp.notion.com/mcp",
    "github": "https://api.githubcopilot.com/mcp/",
    "slack": "https://mcp.slack.com/mcp",
}
TRUST_WEIGHTS = {"gdrive":1.0,"notion":1.0,"github":0.85,"slack":0.85,"web":0.60}

class MCPManager:
    def __init__(self, enabled_sources: list[str], encrypted_tokens: dict, fernet_key: bytes):
        self.sources = enabled_sources
        fernet = Fernet(fernet_key)
        self.tokens = {k: fernet.decrypt(v.encode()).decode() for k,v in encrypted_tokens.items() if k in enabled_sources}

    async def search(self, topic: str) -> list[dict]:
        tasks = [self._query_source(src, topic) for src in self.sources]
        results = await asyncio.gather(*tasks, return_exceptions=True)
        flat = [item for sublist in results if isinstance(sublist, list) for item in sublist]
        return self._deduplicate(flat)

    async def _query_source(self, source: str, topic: str) -> list[dict]:
        if source == "web":
            return await self._web_search(topic)
        try:
            client = anthropic.Anthropic()
            resp = client.beta.messages.create(
                model="claude-sonnet-4-20250514", max_tokens=2000,
                mcp_servers=[{"type":"url","url":MCP_URLS[source],"name":source,"authorization_token":self.tokens.get(source,"")}],
                messages=[{"role":"user","content":f"Search for and retrieve all content relevant to: {topic}. Return the raw text content found."}],
                betas=["mcp-client-2025-04-04"]
            )
            text = " ".join([b.text for b in resp.content if hasattr(b,"text")])
            return [{"text":text,"source":source,"url":"","trust":TRUST_WEIGHTS[source]}]
        except Exception as e:
            return []

    async def _web_search(self, topic: str) -> list[dict]:
        client = anthropic.Anthropic()
```



```

resp = client.messages.create(
    model="claude-sonnet-4-20250514", max_tokens=1500,
    tools=[{"type": "web_search_20250305", "name": "web_search", "max_uses": 3}],
    messages=[{"role": "user", "content": f"Search for educational content about:
{topic}. Focus on reliable sources like Wikipedia, MDN, GeeksForGeeks."}]
)
text = " ".join([b.text for b in resp.content if hasattr(b, "text")])
return [{"text": text, "source": "web", "url": "", "trust": 0.60}]

def _deduplicate(self, items: list[dict]) -> list[dict]:
    seen, result = set(), []
    for item in items:
        h = hashlib.sha256(item["text"][:200].encode()).hexdigest()
        if h not in seen:
            seen.add(h); result.append(item)
    return result

```

2. Add to users collection schema: `enabled_mcp_sources: List[str] = []`, `mcp_tokens: dict = {}`
3. POST `/mcp/connect/:source` – for now implement simple token storage:
 Body: `{token: str}` (user pastes their API key/token)
 Encrypt with Fernet using `ENCRYPTION_KEY` from env, store in `user.mcp_tokens[source]`
 Set source in `user.enabled_mcp_sources`
 Return `{connected: true, source: source}`
4. GET `/mcp/sources` – return all 5 sources with `{name, connected: bool}` for authenticated user
5. DELETE `/mcp/disconnect/:source` – remove from `mcp_tokens` and `enabled_mcp_sources`
6. UPDATE POST `/explain/generate`:
 - Load user's `enabled_mcp_sources` and `mcp_tokens` from DB
 - If any sources enabled: `mcp = MCPManager(sources, tokens, FERNET_KEY)`; `docs = await mcp.search(topic)`
 - Format docs as: `"\n\nKnowledge from your sources:\n" + "\n---\n"`.join(`[f"[{d['source']}]: {d['text'][:800]}" for d in docs[:3]]`)
 - Append to prompt before the explanation instruction
 - Save `mcp_sources_used = [d['source'] for d in docs]` to history document

FRONTEND:

1. `/settings/sources` page: show 5 source cards (Google Drive, Notion, GitHub, Slack, Web Search)
 Each card: name, description, status badge (Connected / Not connected in different colours)
 Connect form: text input for token/key, Connect button → POST `/mcp/connect/:source`
 Disconnect button → DELETE `/mcp/disconnect/:source`
 Toggle to enable/disable without disconnecting
2. `ExplainCard`: add collapsible "Sources used" section showing chip per source with source name
 Only show if `mcp_sources_used` is non-empty in response

For Phases 2, 4, 6, 7, 8: generate agent prompts by combining the task list from Section 11 (Phase Roadmap) with the relevant schema fields from Section 06 and endpoint specs from Section 07. Follow the same format as the prompts above.

14 — Testing Strategy

Backend Unit Tests (pytest)

- `test_style_weights_update()`: feed score=1 for analogy → assert analogy weight increased, all weights sum to 1.0
- `test_style_weights_normalisation()`: set one weight to 10.0, update, assert all weights ≤ 1.0 and sum = 1.0
- `test_mcp_deduplication()`: feed two identical chunks from different sources → assert only one in output
- `test_rag_chunking()`: 1200-word document → assert ≥ 2 chunks, all chunks ≤ 500 tokens
- `test_jwt_expiry()`: expired token in Authorization header → assert 401 response
- `test_confusion_detector()`: "I don't understand what you mean" → assert `confusion_score` > 0.7
- `test_trust_weighting()`: gdrive chunk score 0.75, web chunk score 0.80 → gdrive ranks higher ($0.75 \times 1.0 > 0.80 \times 0.60$)

Backend Integration Tests (pytest + httpx AsyncClient)

- `test_full_explain_flow()`: register user → login → POST `/explain/generate` → assert 200 + explanation field present
- `test_auth_flow()`: register → login → use token → refresh → logout → old token rejected
- `test_feedback_updates_weights()`: generate → rate thumbs up → GET `/profile` → assert rated style weight > 0.34
- `test_mcp_connect_disconnect()`: POST `/mcp/connect/gdrive` with test token → GET `/mcp/sources` shows connected → DELETE `/mcp/disconnect/gdrive` → shows disconnected
- `test_multi_style()`: POST `/explain/multi-style` → assert all three style keys present in response
- `test_history_pagination()`: generate 25 explanations → GET `/profile/history?limit=20` → assert 20 returned + `has_more=true`

Frontend Tests (Vitest + React Testing Library)

- Topic input + Submit → loading spinner appears → explanation renders after mock API response
- Style tab switching in multi-style mode: click each tab → correct explanation shown
- Thumbs up click → POST `/feedback/rate` called with score:1 (verify with msw mock)
- Profile page: style_weights bars render with correct widths proportional to weight values
- Unauthenticated user visiting `/profile` → redirected to `/login`
- AudioPlayer: play button toggles to pause, speed selector changes playback rate attribute

End-to-End Tests (Playwright)

- Full flow: register → login → type "quicksort" → submit → explanation appears → thumbs up → profile weights changed
- Multi-style: generate → click Code tab → see code explanation → rate it → switch to Analogy → rate differently
- MCP: connect source (mock OAuth) → generate explanation → see source citation chip in Sources section

- Chat mode: generate → click chat → ask "can you give me a simpler example?" → get relevant follow-up
- Audio: generate → click Audio button → audio player appears with playback controls

Manual QA Checklist (run at end of each phase)

- Phase 1: explanation received in < 10s, all 3 styles produce different output for same topic
- Phase 2: JWT stored correctly, page refresh keeps user logged in, logout clears state
- Phase 3: rate 5 analogy explanations → auto-selected style on next login is analogy
- Phase 4: all 3 multi-style tabs populated simultaneously (not sequentially)
- Phase 5: connect Drive → upload PDF → ask topic from PDF → PDF cited in explanation
- Phase 6: add new file to connected source → Celery indexes it → appears in /mcp/indexed-docs
- Phase 7: audio plays within 10s of clicking, Mermaid diagram renders without errors
- Phase 8: dependency graph shows prerequisites for a complex topic like "neural networks"

15 — Deployment Plan & Environment Variables

All Environment Variables (.env)

Variable	Example Value	Purpose
ANTHROPIC_API_KEY	sk-ant-...	Claude API key — primary LLM and MCP server connections
OPENAI_API_KEY	sk-...	Embeddings (text-embedding-3-small), TTS, Whisper, DALL-E
MONGODB_URI	mongodb+srv://...	MongoDB Atlas connection string
JWT_SECRET	random 64-char hex	Access token signing secret. Generate: openssl rand -hex 32
JWT_REFRESH_SECRET	random 64-char hex	Refresh token signing secret. Different from JWT_SECRET.
ENCRYPTION_KEY	Fernet.generate_key()	AES-128 key for MCP token encryption. Generate once, never change.
REDIS_URL	redis://...	Upstash Redis URL. Used for Celery broker and caching.
CHROMADB_HOST	localhost	ChromaDB host for dev. Use Pinecone in prod.
PINECONE_API_KEY	...	Pinecone API key for production vector store
PINECONE_INDEX_NAME	elim-embeddings	Pinecone index name. Create with 1536 dimensions, cosine metric.
S3_BUCKET_NAME	elim-audio	S3/R2 bucket for TTS audio files
S3_ACCESS_KEY	...	AWS/R2 access key for audio file uploads
S3_SECRET_KEY	...	AWS/R2 secret key
S3_ENDPOINT_URL	https://...r2.cloudflarestorage.com	Cloudflare R2 endpoint (leave blank for AWS S3)
ELEVENLABS_API_KEY	...	ElevenLabs TTS API key (Phase 7)
SENTRY_DSN_BACKEND	https://...	Sentry DSN for FastAPI error tracking
SENTRY_DSN_FRONTEND	https://...	Sentry DSN for React error tracking
LANGCHAIN_API_KEY	ls_...	LangSmith API key for prompt tracing
LANGCHAIN_TRACING_V2	true	Enable LangSmith tracing on all LangChain calls
VITE_API_URL	http://localhost:8000	Backend URL for React (dev). Set to Railway URL in prod.

Deployment Steps

- BACKEND: Push to GitHub → Railway auto-deploys on push to main branch
- Set all env vars in Railway dashboard → deploy shows live URL
- Add Railway URL to VITE_API_URL in Vercel env vars

- FRONTEND: Connect GitHub repo to Vercel → auto-deploys on push → set VITE_API_URL
- CELERY: Add Celery worker as separate Railway service from same repo
- MONGODB: Use MongoDB Atlas free tier (M0) — add Railway IP to allowlist
- REDIS: Upstash free tier — copy connection string to REDIS_URL
- CHROMADB → PINECONE: Create Pinecone index (1536 dims, cosine), add API key to env

16 — Resume & Viva Talking Points

These are the exact phrases to use. Know what they mean — every word is backed by something you actually built.

Resume Bullet Points

- "Built an adaptive AI explanation engine using dynamic prompt engineering and user modeling — explanations personalised to individual learning style and difficulty"
- "Implemented a feedback-driven preference weight update loop (weighted moving average) that auto-selects explanation style based on per-user rating history"
- "Designed a modular LangChain prompt pipeline with separate templates for 5 explanation modes (single, multi-style, chat, diagram, Socratic)"
- "Integrated MCP (Model Context Protocol) to retrieve live context from Google Drive, Notion, and GitHub — feeding a RAG pipeline with user-owned study material"
- "Built full RAG pipeline: document chunking, text-embedding-3-small embeddings, ChromaDB/Pinecone vector store, cosine similarity retrieval with source trust weighting"
- "Architected full-stack personalization engine: React 18, FastAPI, MongoDB, Redis, Celery workers, S3 — production-deployed on Vercel + Railway"
- "Implemented 10 unique AI features including Spaced Repetition (SM-2 algorithm), Concept Dependency Graph, Confusion Detector, and Peer Style Matching"

Viva Questions & Answers

Why MCP over direct API calls?

MCP is a standardised protocol — adding a new knowledge source is one config object, not a custom integration. It also handles auth, pagination, and retries. More importantly, it lets the RAG pipeline pull live user-owned data instead of stale indexed content.

How does the feedback loop actually work?

Every explanation saves its style to MongoDB. After rating (thumbs up = +1, thumbs down = -1), I apply: $\text{new_weight} = 0.8 \times \text{old} + 0.2 \times \text{signal}$, then normalise all weights to sum to 1. Over sessions, the highest-rated style's weight drifts toward 1.0. I can demonstrate this live with a graph.

Why is this not just a ChatGPT wrapper?

Three reasons: (1) The prompt is dynamically constructed from a live user profile — no two users get the same prompt. (2) MCP sources mean the context is grounded in the user's actual notes. (3) The feedback loop means the system's behaviour changes over time per user. That's a system, not a wrapper.

How do you prevent hallucination in RAG?

The prompt explicitly says "use ONLY the following context from the user's sources." The LLM is instructed to cite sources inline. The Quality Scorer runs a second LLM call to check accuracy — if score < 3/5, it retries. We never show a low-confidence explanation.

What is the hardest

Parallel MCP queries with proper error isolation — if Notion is down,

technical challenge?

Drive and GitHub should still return results. `asyncio.gather(return_exceptions=True)` handles this. The trust weighting then ensures a partial result from one source doesn't over-weight against the others.

How does the Concept Dependency Graph work?

When a user asks about Topic B, I embed it using `text-embedding-3-small` and compare cosine similarity against all entries in their `topic_history` embeddings. Topics with similarity > 0.8 are related — ones they haven't studied yet are prerequisites. I render these with `react-flow` as a clickable prerequisite graph.

How is user data kept private in Peer Matching?

Clustering is done on `style_weights` vectors only — no topic history, no personal data. Shared explanations are text blobs that were rated highly — never linked to the original user. The cluster ID stored per user is a pseudonymous integer, not a user reference.

ELIM — Explain Like I'm Me

Complete Build Roadmap · All Rights Belong to the Builder
Adaptive AI · Prompt Engineering · RAG · MCP · Multi-Modal · Feedback Loops